MODUL PRAKTIKUM PEMROGRAMAN WEB

MODUL 5: MANIPULASI DOM, EVENT HANDLING & VALIDASI FORM INPUT

Panduan Praktis Interaktivitas Web, Validasi Form Sederhana, dan Penyimpanan Data LocalStorage

BAB 1: PENGENALAN DOM, EVENT & LOCALSTORAGE

1.1 Memilih Elemen HTML (DOM Selection)

Document Object Model (DOM) adalah representasi halaman web yang memungkinkan JavaScript membaca dan mengubah struktur HTML. Untuk memanipulasi elemen, langkah pertama adalah memilih elemen tersebut menggunakan CSS selector:

- **document.querySelector('selector'):** Memilih satu elemen pertama yang cocok dengan selector (contoh: id '#taskInput', class '.btn-add', atau tag 'h2').
- **document.querySelectorAll('selector'):** Memilih seluruh elemen yang cocok dan mengembalikannya dalam bentuk kumpulan elemen (NodeList).

```
// Memilih elemen berdasarkan ID dan Class:
const formTugas = document.querySelector("#todoForm");
const inputTugas = document.querySelector("#taskInput");
const listContainer = document.querySelector("#todoList");
```

1.2 Mengubah Konten dan Kelas CSS

Setelah elemen dipilih, kita dapat mengubah teks, kode HTML di dalamnya, maupun memberikan efek visual melalui kelas CSS:

- **textContent:** Mengubah atau mengambil teks murni di dalam elemen (aman dari celah keamanan).
- **innerHTML:** Mengisi elemen dengan potongan tag HTML baru.
- **classList.add('nama-kelas'):** Menambahkan kelas CSS pada elemen (misalnya memberi border merah saat error).
- **classList.remove('nama-kelas'):** Menghapus kelas CSS tertentu.
- **classList.toggle('nama-kelas'):** Menambah kelas jika belum ada, atau menghapusnya jika sudah ada (sangat cocok untuk mencoret tugas selesai).

```
// Mengubah tampilan elemen:
inputTugas.classList.add("error"); // Beri tanda error
inputTugas.classList.remove("error"); // Hapus tanda error
itemTugas.classList.toggle("completed"); // Ubah status selesai/belum
```

1.3 Menangani Interaksi Pengguna (Event Handling)

Aplikasi web menjadi hidup karena merespons aksi pengguna. Kita menggunakan metode addEventListener untuk memantau kejadian (event) tertentu:

- **Event 'submit':** Terjadi saat formulir dikirimkan (tombol ditekan atau pengguna menekan Enter pada input).
- **Event 'click':** Terjadi ketika elemen (seperti tombol Selesai atau Hapus) diklik.
- **e.preventDefault():** Perintah wajib saat menangani formulir untuk menghentikan perilaku bawaan browser (mencegah halaman web me-refresh secara otomatis).

```
formTugas.addEventListener("submit", (e) => {
    e.preventDefault(); // Mencegah reload halaman
    console.log("Formulir berhasil dikirim tanpa reload!");
});
```

1.4 Validasi Input Form Sederhana

Sebelum data dimasukkan ke dalam daftar, aplikasi perlu memastikan pengguna tidak mengirimkan teks kosong. Validasi sederhana dilakukan dengan memeriksa panjang teks setelah dipotong spasinya (.trim()):

- **input.value.trim():** Mengambil teks input dan membuang spasi kosong di awal dan akhir.
- **Kondisi Validasi:** Jika teks kosong atau kurang dari 3 huruf, tampilkan pesan peringatan dan batalkan penambahan tugas.

```
const teksTugas = inputTugas.value.trim();
if (teksTugas.length < 3) {
    pesanError.style.display = "block"; // Munculkan pesan error
    inputTugas.classList.add("error");
    return; // Berhenti dan jangan lanjutkan proses simpan
}
```

1.5 Menyimpan Data di Browser (LocalStorage)

Secara default, variabel JavaScript akan ter-reset saat browser di-refresh. Agar daftar tugas tetap tersimpan, kita menggunakan LocalStorage. Karena LocalStorage hanya bisa menyimpan teks, data array dikonversi dengan JSON:

- **Menyimpan Data:** localStorage.setItem('todo_tasks', JSON.stringify(tasks));
- **Membaca Data:** const tasks = JSON.parse(localStorage.getItem('todo_tasks')) || [];
- **Menghapus Data:** localStorage.removeItem('todo_tasks');

BAB 2: PANDUAN PRAKTIK TERSTRUKTUR (APLIKASI TO-DO LIST)

Pada bagian praktik ini, kita akan membuat aplikasi To-Do List yang bersih, mudah dipahami, memiliki validasi input, serta menyimpan data secara permanen di LocalStorage.

Langkah 2.1: Menyiapkan Struktur Halaman (index.html)

Buat berkas baru bernama index.html di dalam folder modul-5, lalu ketikkan kode berikut:

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Aplikasi To-Do List - Modul 5</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="todo-app">
        <h2>Daftar Tugas Saya</h2>

        <!-- Form Input Tugas -->
        <form id="todoForm" novalidate>
            <div class="input-section">
                <input type="text" id="taskInput" placeholder="Ketikkan tugas baru
Anda...">
                <button type="submit" class="add-btn">Tambah</button>
            </div>
            <div class="error-message" id="inputError">Tugas tidak boleh kosong atau
kurang dari 3 karakter!</div>
        </form>

        <!-- Daftar Tugas -->
        <ul class="todo-list" id="todoList"></ul>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

Langkah 2.2: Menata Tampilan Antarmuka (style.css)

Buat berkas baru bernama style.css di dalam folder modul-5, lalu ketikkan kode penataan berikut:

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f8fafc; color: #1e293b; padding: 40px 20px; display: flex;
justify-content: center;
}
.todo-app {
    background: #ffffff; width: 100%; max-width: 500px; padding: 24px; border-radius:
12px;
    border: 1px solid #e2e8f0; box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
}
h2 { color: #1d4ed8; margin-bottom: 20px; text-align: center; font-size: 22px; }
.input-section { display: flex; gap: 10px; margin-bottom: 8px; }
input[type="text"] {
```

```
    flex: 1; padding: 10px 14px; border: 1px solid #cbd5e1; border-radius: 6px; font-
size: 14px; outline: none;
}
input[type="text"].error { border-color: #dc3545; background-color: #fff5f5; }
button.add-btn {
    background-color: #1d4ed8; color: white; border: none; padding: 10px 18px;
    border-radius: 6px; font-weight: bold; cursor: pointer;
}
button.add-btn:hover { background-color: #1e40af; }
.error-message { color: #dc3545; font-size: 12px; margin-bottom: 14px; display: none;
}
.todo-list { list-style: none; margin-top: 14px; }
.todo-item {
    display: flex; align-items: center; justify-content: space-between;
    padding: 10px 12px; background-color: #f1f5f9; border-radius: 6px; margin-bottom:
8px;
}
.todo-item.completed { border-left-color: #16a34a; opacity: 0.7; }
.todo-item.completed .todo-text { text-decoration: line-through; color: #64748b; }
.todo-text { font-size: 14px; flex: 1; margin-right: 10px; }
.todo-actions { display: flex; gap: 6px; }
.action-btn {
    border: none; padding: 5px 9px; border-radius: 4px; font-size: 12px; cursor:
pointer; font-weight: bold;
}
.complete-btn { background-color: #16a34a; color: white; }
.complete-btn:hover { background-color: #15803d; }
.delete-btn { background-color: #dc2626; color: white; }
.delete-btn:hover { background-color: #b91c1c; }
```

Langkah 2.3: Menulis Kode Logika Sederhana (script.js)

Buat berkas baru bernama script.js di dalam folder modul-5, lalu ketikkan kode JavaScript yang bersih berikut:

```
// File: script.js
document.addEventListener("DOMContentLoaded", () => {
    // 1. Ambil elemen DOM
    const todoForm = document.querySelector("#todoForm");
    const taskInput = document.querySelector("#taskInput");
    const inputError = document.querySelector("#inputError");
    const todoList = document.querySelector("#todoList");

    // 2. Fungsi membaca dan menyimpan ke LocalStorage
    const getTasks = () => JSON.parse(localStorage.getItem("todo_tasks")) || [];
    const saveTasks = (tasks) => localStorage.setItem("todo_tasks",
JSON.stringify(tasks));

    // 3. Fungsi menampilkan daftar tugas ke layar
    const renderTasks = () => {
        const tasks = getTasks();
        todoList.innerHTML = "";
```

```
        if (tasks.length === 0) {
            todoList.innerHTML = "<p style='text-align:center; color:#64748b; font-
size:13px; padding:10px;'>Belum ada tugas tersimpan.</p>";
            return;
        }

        tasks.forEach((task, index) => {
            const li = document.createElement("li");
            li.className = `todo-item ${task.completed ? 'completed' : ''}`;

            li.innerHTML = `
                <span class="todo-text">${task.text}</span>
                <div class="todo-actions">
                    <button class="action-btn complete-btn" data-index="${index}">
                        ${task.completed ? 'Batal' : 'Selesai'}
                    </button>
                    <button class="action-btn delete-btn" data-
index="${index}">Hapus</button>
                </div>
            `;
            todoList.appendChild(li);
        });
    };

    // 4. Tambah tugas saat formulir dikirim
    todoForm.addEventListener("submit", (e) => {
        e.preventDefault();
        const taskText = taskInput.value.trim();

        // Validasi input sederhana
        if (taskText.length < 3) {
            taskInput.classList.add("error");
            inputError.style.display = "block";
            return;
        }

        // Jika valid, hilangkan pesan error
        taskInput.classList.remove("error");
        inputError.style.display = "none";

        // Simpan tugas baru ke array LocalStorage
        const tasks = getTasks();
        tasks.push({ text: taskText, completed: false });
        saveTasks(tasks);

        taskInput.value = "";
        renderTasks();
    });

    // 5. Tangani klik tombol Selesai dan Hapus menggunakan Event Listener pada
todoList
```

```
    todoList.addEventListener("click", (e) => {
        const index = e.target.dataset.index;
        if (index === undefined) return;

        const tasks = getTasks();

        if (e.target.classList.contains("complete-btn")) {
            tasks[index].completed = !tasks[index].completed;
            saveTasks(tasks);
            renderTasks();
        } else if (e.target.classList.contains("delete-btn")) {
            tasks.splice(index, 1);
            saveTasks(tasks);
            renderTasks();
        }
    });

    // Render pertama kali saat halaman selesai dimuat
    renderTasks();
});
```

Langkah 2.4: Uji Coba & Simpan ke GitHub

1. Buka index.html di browser. Coba klik 'Tambah' dalam keadaan kosong untuk memastikan pesan error muncul.

2. Masukkan 2–3 tugas baru, lalu uji tombol 'Selesai' (tugas akan dicoret) dan tombol 'Hapus'.

3. Muat ulang halaman (Refresh / F5) untuk memastikan tugas tetap ada dari LocalStorage.

4. Simpan pembaruan ke Git dengan perintah terminal:

```
git status
git add .
git commit -m "feat: menyelesaikan praktik modul 5 todo list dengan localstorage"
git branch -M pertemuan-5
git push -u origin pertemuan-5
```

BAB 3: TUGAS MANDIRI

- **Fitur 1 - Penghitung Jumlah Tugas (Task Counter):** Tambahkan elemen teks di bawah judul atau di atas daftar tugas untuk menampilkan ringkasan jumlah tugas secara dinamis. Contoh format: 'Total: 3 Tugas | Selesai: 1 | Belum Selesai: 2'. Angka ini harus otomatis diperbarui setiap kali tugas ditambah, dicoret, atau dihapus.
- **Fitur 2 - Tombol 'Bersihkan Tugas Selesai':** Tambahkan satu tombol bertuliskan 'Bersihkan Tugas Selesai' di bagian bawah daftar tugas. Ketika tombol ini diklik, aplikasi hanya menghapus tugas-tugas yang berstatus completed: true dari LocalStorage, sementara tugas yang belum selesai tetap tersimpan dengan aman.

- **Fitur 3 - Konfirmasi Sebelum Menghapus (Confirm Dialog):** Saat pengguna mengklik tombol 'Hapus' pada salah satu tugas, tampilkan dialog konfirmasi bawaan peramban menggunakan confirm('Apakah Anda yakin ingin menghapus tugas ini?'). Jika pengguna menekan 'OK', barulah tugas dihapus; jika menekan 'Cancel', tugas batal dihapus.

Instruksi Pengumpulan Git

Setelah menyelesaikan dan menguji ketiga fitur di atas, jalankan perintah Git berikut:

```
git status
git add .
git commit -m "feat(modul-5): menambahkan counter, clear completed, dan konfirmasi
hapus"
git push -u origin pertemuan-5
```